CMPUT 654: Mathematical Foundations of Modern AI Systems Fall 2026

Lecture 1: Decoder transformers and autoregressive inference

September 1, 2026

Lecturer: Csaba Szepesvári *Note prepared by: Csaba Szepesvári*

Purpose of this note. This is an example of the expected scribe-note style: record what happened in the lecture, define the mathematical objects precisely, and distinguish the main calculation from explanatory remarks. It is an exposition, not a transcript.

Delivery note. Administrative material occupied a substantial part of the meeting. The lecture opened with the harness surrounding a language model and then zoomed in on tokenization and how a decoder transformer performs inference. Training was described verbally. The lecture also discussed the quadratic cost of attention, key–value caching, maximum context length, and limitations of attention on long contexts. No training objective, loss, or gradient equation was presented. The motivation for learning and the first learning-theoretic results were postponed to Lecture 2.

1.1 The model inside a harness

A deployed AI system is generally larger than one call to a neural network. A *harness* is the surrounding program that constructs the model's context, calls the model, interprets the result, optionally invokes tools, records the new observation, and decides whether to call the model again. It may provide instructions, conversation history, retrieved documents, external memory, or tool outputs. Schematically, one pass through the loop is

$$\boxed{\text{task state}} \xrightarrow{\text{harness}} \boxed{s \in \Sigma^*} \xrightarrow{\text{tokenizer } \tau} \boxed{x_{<t} \in [V]^*}, \tag{1.1}$$

followed by

$$\boxed{x_{<t}} \xrightarrow{\text{model}} \boxed{p_\theta(\,\cdot \mid x_{<t})} \xrightarrow{\text{decoding}} \boxed{\text{next token or action}}. \tag{1.2}$$

Output tokens may be detokenized into text or parsed as a tool call. The resulting observation updates the task state, and the loop may begin again. A harness can use several model calls before returning an answer. The behavior of the complete system can therefore depend on the tokenizer, model parameters, decoding rule, maintained state, tools, and control loop. The rest of this note expands the tokenizer and model boxes.

1.2 Tokenization and the modeled sequence

The word *tokenization* comes from the compiler pipeline. A compiler's lexer maps a character stream to tokens such as identifiers, numerals, and operators using a language specification. An LLM tokenizer has the same broad string-to-sequence interface, but its vocabulary and segmentation rules are normally constructed from a text corpus rather than from the grammar of a programming language.

Let Σ be a finite base alphabet and let

$$\mathcal{V} = \{v_1, \dots, v_V\} \subset \Sigma^+ \tag{1.3}$$

be a vocabulary of nonempty strings. A lossless tokenizer maps

$$\tau : \Sigma^* \longrightarrow [V]^*, \qquad \tau(s) = (x_1, \ldots, x_T), \qquad s = v_{x_1} v_{x_2} \cdots v_{x_T}. \tag{1.4}$$

Detokenization concatenates the vocabulary strings. Because every vocabulary item is nonempty,

$$|s| = \sum_{t=1}^{T} |v_{x_t}| \geq T. \tag{1.5}$$

Thus tokenization, understood as segmentation over a fixed base alphabet, cannot increase sequence length and decreases it whenever at least one chosen token contains more than one base symbol. Normalization, conversion from Unicode characters to bytes, and the addition of special tokens may change the sequence before or after this segmentation; those are separate operations.

Practical tokenizers may also normalize text, first split it into coarse pieces, or begin from bytes to guarantee coverage. Tokens often coincide with useful linguistic units, but this is not guaranteed and is not the defining objective. Appendix A gives optional background on how tokenizers may be constructed; those details were not part of the lecture.

Once the tokenizer is fixed, a stochastic language or sequence model specifies a stochastic process $(X_t)_{t \geq 1}$ whose state space is $[V]$. An autoregressive model gives a conditional distribution for the next token after every prefix. For a fixed token sequence,

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}). \tag{1.6}$$

A beginning-of-sequence convention initializes the process; an end-of-sequence token can represent variable-length strings.

1.3 The inference problem

Given a token prefix $x_{<t} = (x_0, \ldots, x_{t-1})$, a language model returns a categorical distribution for the next token:

$$x_{<t} \longmapsto p_\theta(\cdot \mid x_{<t}). \tag{1.7}$$

An inference algorithm selects x_t from this distribution, appends it to the prefix, and repeats. Thus the model and the rule used to decode from it are distinct objects.

Orientation convention. On the board, the sequence was drawn from left to right:

$$\boxed{x_0} \quad \boxed{x_1} \quad \cdots \quad \boxed{x_{T-1}}. \tag{1.8}$$

In the matrix equations below, each position is instead represented by one *row*, and positions are stacked from top to bottom. Columns hold vocabulary coordinates, model features, head features, or source positions, depending on the matrix. Thus the orientation of a board diagram is not the storage convention of the equations. We state what the rows and columns index whenever a new kind of matrix appears.

Let the vocabulary have size V, and identify token $x_t \in [V]$ with the standard basis vector $e_{x_t} \in \{0,1\}^V$. Include a beginning-of-sequence token x_0. For T prediction positions, define $Z \in \{0,1\}^{T \times V}$ by

$$Z_{t,:} = e_{x_{t-1}}^\top, \qquad t = 1, \ldots, T. \tag{1.9}$$

Rows of Z index sequence positions and columns index vocabulary items: $Z_{t,v} = 1$ precisely when $x_{t-1} = v$. With an embedding matrix $E \in \mathbb{R}^{V\times d}$ and additive absolute-position vectors $P \in \mathbb{R}^{T\times d}$, the input is

$$H^0 = ZE + P, \qquad H^0_{t,:} = E_{x_{t-1},:} + P_{t,:}. \tag{1.10}$$

Rows of E index vocabulary items and columns index the d model features; row v is the embedding of token v. Rows of P and H^0 index positions, and their columns index model features. The one-hot notation makes the map explicit; an implementation performs a row lookup in E.

The complete inference calculation can be summarized by the following block diagram:

$$\boxed{x_{<t}} \longrightarrow \boxed{ZE + P} \longrightarrow \boxed{H^1 \longrightarrow \cdots \longrightarrow H^L} \longrightarrow \boxed{z_t} \longrightarrow \boxed{p_\theta(\,\cdot \mid x_{<t})}. \tag{1.11}$$

The next sections expand the three middle boxes.

Where is position added? Equation (1.10) describes additive absolute positions. They are added once, before the first transformer block; they are not added again in every block. Other position mechanisms enter the computation differently. For example, a layer- and head-specific relative-position bias changes the attention calculation to

$$A_a = \mathrm{softmax}_{\mathrm{row}}\left(\frac{Q_a K_a^\top}{\sqrt{d_h}} + M + B_{\ell,a}\right), \qquad (B_{\ell,a})_{ij} = b_{\ell,a}\big(\rho(i-j)\big). \tag{1.12}$$

Here ρ may clip or bucket the relative displacement $i - j$, and $b_{\ell,a}$ assigns a learned scalar to each bucket. T5 uses this form with the bias parameters shared across layers and different across heads. ALiBi instead uses a fixed, head-dependent linear penalty such as $-m_a(i-j)$ for a causal pair $j \le i$. RoPE applies a position-dependent rotation to queries and keys in each attention layer that uses RoPE. These are alternative architectural choices; the symbol P in (1.10) does not also stand for relative bias or RoPE. In (1.12), rows index query positions and columns index key positions, so $i - j$ compares the output position i with the source position j.

Remark. The relative-displacement calculation for rotary positions will be developed separately in the homework.

1.4 One fully specified decoder block

Suppose $H^\ell \in \mathbb{R}^{T\times d}$ enters layer ℓ. The lecture described the attention calculation but did not state that a standard transformer uses multiple attention heads. We include that correction here and specify a pre-normalized block with h heads of width d_h. In H^ℓ, row i is the activation at sequence position i, and column r is model-feature coordinate r:

$$H^\ell = \begin{bmatrix} h_1^\ell \\ \vdots \\ h_T^\ell \end{bmatrix} \in \mathbb{R}^{T\times d}, \qquad h_i^\ell = H^\ell_{i,:} \in \mathbb{R}^{1\times d}. \tag{1.13}$$

First define the normalization map. For $u \in \mathbb{R}^d$, let

$$\mu(u) = \frac{1}{d}\sum_{r=1}^{d} u_r, \qquad \sigma^2(u) = \frac{1}{d}\sum_{r=1}^{d}(u_r - \mu(u))^2, \tag{1.14}$$

and, for learned $\gamma_{\ell,m}, \beta_{\ell,m} \in \mathbb{R}^d$ and fixed $\varepsilon > 0$, define

$$\mathrm{LayerNorm}_{\ell,m}(u) = \gamma_{\ell,m} \odot \frac{u - \mu(u)\mathbf{1}}{\sqrt{\sigma^2(u) + \varepsilon}} + \beta_{\ell,m}, \qquad m \in \{1, 2\}. \tag{1.15}$$

On a matrix, $\mathrm{LayerNorm}_{\ell,m}$ acts row by row: it normalizes the d feature coordinates at each position independently. Layer normalization was not discussed in the lecture; it is included so that the displayed block is fully specified.

At the block-diagram level, a pre-normalized decoder layer performs two residual calculations:

$$H^\ell \longrightarrow \boxed{\text{LayerNorm} \longrightarrow \text{masked multi-head attention}} \xrightarrow{+H^\ell} G, \tag{1.16}$$

$$G \longrightarrow \boxed{\text{LayerNorm} \longrightarrow \text{MLP or MoE}} \xrightarrow{+G} H^{\ell+1}. \tag{1.17}$$

The equations below specify the calculations inside these boxes.

For head $a \in [h]$, let

$$U = \mathrm{LayerNorm}_{\ell,1}(H^\ell), \tag{1.18}$$

$$Q_a = UW^Q_{\ell,a}, \qquad K_a = UW^K_{\ell,a}, \qquad V_a = UW^V_{\ell,a}, \tag{1.19}$$

$$A_a = \mathrm{softmax}_{\mathrm{row}}\left(\frac{Q_a K_a^\top}{\sqrt{d_h}} + M\right), \qquad O_a = A_a V_a, \tag{1.20}$$

where

$$W^Q_{\ell,a}, W^K_{\ell,a}, W^V_{\ell,a} \in \mathbb{R}^{d \times d_h}, \qquad M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases} \tag{1.21}$$

The matrices U, Q_a, K_a, V_a, and O_a all have one row per sequence position. The columns of U are model features; the columns of $Q_a, K_a, V_a, O_a \in \mathbb{R}^{T \times d_h}$ are head features. The score, mask, and attention matrices lie in $\mathbb{R}^{T \times T}$. Their row i refers to query or output position i, and their column j refers to key/value or source position j. In particular, $(A_a)_{ij}$ is the weight with which position i uses the value at position j.

The softmax acts row by row. The causal mask makes the attention weight of every future source position exactly zero. Concatenate the head outputs and apply the first residual connection:

$$G = H^\ell + [O_1 \mid \cdots \mid O_h]W^O_\ell, \qquad W^O_\ell \in \mathbb{R}^{hd_h \times d}. \tag{1.22}$$

The notation $[O_1 \mid \cdots \mid O_h] \in \mathbb{R}^{T \times hd_h}$ concatenates the *columns*: rows remain sequence positions, while the columns collect the features from all heads. Multiplication by W^O_ℓ maps those columns back to d model features, so $G \in \mathbb{R}^{T \times d}$ can be added to H^ℓ. The positionwise feed-forward sublayer and second residual connection give

$$R = \mathrm{LayerNorm}_{\ell,2}(G), \tag{1.23}$$

$$H^{\ell+1} = G + \phi\Big(RW^1_\ell + \mathbf{1}(b^1_\ell)^\top\Big) W^2_\ell + \mathbf{1}(b^2_\ell)^\top, \tag{1.24}$$

with

$$W^1_\ell \in \mathbb{R}^{d \times d_{\mathrm{ff}}}, \quad W^2_\ell \in \mathbb{R}^{d_{\mathrm{ff}} \times d}, \quad b^1_\ell \in \mathbb{R}^{d_{\mathrm{ff}}}, \quad b^2_\ell \in \mathbb{R}^d. \tag{1.25}$$

Here $R \in \mathbb{R}^{T \times d}$ still has positions in rows. The product $RW^1_\ell \in \mathbb{R}^{T \times d_{\mathrm{ff}}}$ replaces the model-feature columns by feed-forward-unit columns, and multiplication by W^2_ℓ returns to d model-feature

columns. The vector $\mathbf{1} \in \mathbb{R}^T$ copies each bias into every position row. For example, $\phi(z) = z\Phi(z)$ coordinatewise is GELU, where Φ is the standard normal cumulative distribution function.

The lecture also mentioned *mixture-of-experts* layers. They replace the single dense feed-forward map by expert maps $f_1, \ldots, f_m$ and a router. For one position row $r \in \mathbb{R}^{1\times d}$, with each $f_e(r) \in \mathbb{R}^{1\times d}$, a sparse version has the form

$$f_{\mathrm{MoE}}(r) = \sum_{e\in S(r)} \rho_e(r) f_e(r), \tag{1.26}$$

where $S(r)$ is a small router-selected subset of experts and the router weights $\rho_e(r)$ are normalized over that subset. This replacement changes the positionwise sublayer, not the attention calculation.

Practical models also change normalization, use gated MLPs, reorganize attention heads, and handle positions differently. The equations specify one complete pedagogical block, not the unique modern design.

The key invariant is causal dependence: after L blocks, row $h_t^L = H_{t,:}^L$ depends only on $x_{<t}$.

1.5 From a hidden vector to the next-token distribution

Let $W_U \in \mathbb{R}^{d\times V}$ and $b_U \in \mathbb{R}^V$. The final row is converted to logits and then to probabilities:

$$z_t = h_t^L W_U + b_U, \tag{1.27}$$

$$p_\theta(x_t = v \mid x_{<t}) = \frac{\exp(z_{t,v})}{\sum_{u=1}^V \exp(z_{t,u})}. \tag{1.28}$$

The row vector $h_t^L \in \mathbb{R}^{1\times d}$ has one entry per model feature. Rows of W_U index those model features and columns index vocabulary items. Consequently, $z_t \in \mathbb{R}^{1\times V}$ has one logit in each vocabulary column, and the softmax turns those columns into next-token probabilities. Weight tying sets $W_U = E^\top$; it is common but not logically required.

The single-token path is therefore

$$e_{x_{t-1}} \longrightarrow E_{x_{t-1},:} \longrightarrow h_t^L \longrightarrow z_t \longrightarrow p_\theta(\,\cdot \mid x_{<t}). \tag{1.29}$$

The contextual vector h_t^L also depends on the earlier tokens through the masked attention blocks.

1.6 Autoregressive generation

A sampled decoder draws

$$X_t \sim p_\theta(\,\cdot \mid X_{<t}), \tag{1.30}$$

whereas greedy decoding chooses

$$X_t \in \arg\max_v p_\theta(v \mid X_{<t}). \tag{1.31}$$

In either case the selected token is appended and the procedure repeats until a stopping rule fires.

Greedy decoding is neither sampling from the model nor, in general, global optimization of the probability of the complete sequence. A locally most probable token can lead to poor later continuations.

There are narrow cases in which greedy decoding is justified. If the system must make exactly one token-valued decision and incurs zero–one loss, an argmax token is Bayes optimal. Greedy decoding is also harmless when every conditional distribution is effectively deterministic, or when a special problem has a greedy-choice property. None of these conditions holds in general for open-ended generation.

Practical note. For open-ended multi-token generation, forget greedy decoding as a default: it is a bad idea. It throws away nearly all of the predictive distribution at every step, and a locally best token can irreversibly eliminate much better continuations.

Remark. A quantitative counterexample to repeated tokenwise argmax will be developed separately in the homework.

1.7 Attention cost and the key–value cache

Write $h_i^\ell = H_{i,:}^\ell$ for the *layer-position activation*: the residual-stream vector at position i after layer ℓ. Computing the output for position i in one causal attention layer compares its query with i keys. Consequently, a full forward pass on a fixed length-t prefix performs

$$\sum_{i=1}^{t} i = \frac{t(t+1)}{2} \tag{1.32}$$

query–key comparisons per head. Both $QK^\top$ and the multiplication of the resulting attention matrix by V therefore require order $t^2 d_h$ arithmetic. A straightforward implementation also materializes a $t \times t$ attention matrix. The causal mask removes the upper triangle, changing a constant but not the quadratic dependence on t.

The first full forward pass over a supplied prompt is called *prefill*. After prefill, a *decode step* generates one new token and advances only its layer-position activations through the network. These two phases have different computational profiles.

Autoregressive decoding has additional structure. At step t and in each layer, only h_t^ℓ, the activation of the new token, must be advanced. Let its query, key, and value be q_t, k_t, v_t. The attention operation for the new row is

$$\alpha_t = \operatorname{softmax}\left(\frac{q_t K_{\le t}^\top}{\sqrt{d_h}}\right), \qquad o_t = \alpha_t V_{\le t}. \tag{1.33}$$

Here $q_t \in \mathbb{R}^{1\times d_h}$ is the new-position row, $K_{\le t}, V_{\le t} \in \mathbb{R}^{t\times d_h}$ have cached positions in rows and head coordinates in columns, $\alpha_t \in \mathbb{R}^{1\times t}$ has one weight per cached source position, and $o_t \in \mathbb{R}^{1\times d_h}$ is the new output row. The earlier keys and values are therefore stored in a *KV cache*. One cached decode step uses order td_h attention arithmetic per head, and after t positions the cache stores order td_h numbers per head and layer.

Without a cache, step t recomputes the complete length-t forward pass, including all earlier layer-position activations, so the attention work over T generated positions is

$$\sum_{t=1}^{T} \Theta(t^2 d_h) = \Theta(T^3 d_h). \tag{1.34}$$

With the cache, only the new row is computed at each step, and the total is

$$\sum_{t=1}^{T} \Theta(t d_h) = \Theta(T^2 d_h). \tag{1.35}$$

Thus caching changes the asymptotic attention cost of sequential generation. A single parallel full-sequence forward pass remains quadratic.

Why are queries not cached? The query q_i was used to compute the output at position i. A future position $t > i$ forms a new query q_t and compares it with the old key k_i; the resulting weight multiplies the old value v_i. Thus future computation reuses k_i and v_i, but never q_i. Caching old queries would consume memory without avoiding future computation.

1.8 Context length and a bounded-score limitation

A deployed model has a *maximum context length*: an upper bound on the number of prompt and generated tokens available to one model call. Position handling, training range, cache memory, attention cost, and implementation choices can all contribute to this limit. The ability to accept T tokens does not guarantee that the model uses every one of them effectively.

There is also a simple mathematical pressure against sharply selecting one position from an increasingly long context. For one query attending to T keys, write

$$s_j = \frac{q^\top k_j}{\sqrt{d_h}}, \qquad \alpha_j = \frac{e^{s_j}}{\sum_{r=1}^{T} e^{s_r}}. \tag{1.36}$$

If the scores remain in a fixed bounded range as T grows, no individual softmax weight can remain sharply concentrated. A quantitative upper bound, its proof, and the corresponding entropy consequence will be developed separately in the homework.

The bounded-score premise can fail: a model can make score gaps grow with context length, and multiple heads and layers can implement more complicated selection mechanisms. Thus the observation does not by itself show that transformers cannot use long contexts.

Entropy language. For a probability vector $\alpha \in \mathbb{R}^T$, its entropy is

$$\mathsf{H}(\alpha) = -\sum_{j=1}^{T} \alpha_j \log \alpha_j. \tag{1.37}$$

It is 0 for a deterministic distribution and $\log T$ for the uniform distribution. Thus small entropy describes concentrated attention and large entropy describes diffuse attention.

Related empirical phenomenon. The terms *lost in the middle* and *context rot* refer to observed failures to use long contexts uniformly or reliably, sometimes well below a model's advertised maximum length. These experiments concern complete trained systems and task performance. They are related to the question above, but the bounded-score observation alone does not explain them.

1.9 What was said about training

Training was described only in words. The system is shown many text prefixes together with the tokens that actually followed them. Its predictions are compared with those next tokens, and the shared weights are modified to improve the predictions across the data. Lecture 3 will express this description using probabilistic models, log loss, maximum likelihood, KL divergence, and compression.

1.10 What exactly is contained in the parameter vector?

The symbol θ collects the trainable numerical parameters of the conditional model. For the particular architecture displayed in this note, and assuming that the additive position vectors are learned, one explicit accounting is

$$\theta = \Big(E, P, W_U, b_U, \{W^Q_{\ell,a}, W^K_{\ell,a}, W^V_{\ell,a}\}_{\ell,a}, \{W^O_\ell, W^1_\ell, W^2_\ell, b^1_\ell, b^2_\ell, \gamma_{\ell,1}, \beta_{\ell,1}, \gamma_{\ell,2}, \beta_{\ell,2}\}_\ell\Big). \quad (1.38)$$

With weight tying, $W_U = E^\top$ and is not an independent parameter. Fixed position maps and the causal mask M are not learned parameters. An MoE layer replaces the dense-MLP entries above by the expert and router parameters. The token vocabulary and segmentation rules are normally constructed before neural-network training and held fixed. They determine V, the input sequences, and the shapes of E and W_U, but they are not entries of θ in this account.

The token prefix, hidden states, queries, keys, values, attention weights, logits, and KV cache are computed quantities, not parts of θ. The decoding rule and the surrounding harness are also outside θ. Thus training the model means choosing the shared numerical parameters in this display; specifying the deployed AI system requires additional choices.

1.11 Take-home messages

- A deployed AI system places the language model inside a "harness" that constructs context, can maintain state and invoke tools, and may make repeated model calls.
- A lossless tokenizer segments a sequence over a base alphabet into nonempty vocabulary strings. Its token sequence cannot be longer than the base-symbol sequence and is shorter whenever a selected token contains several base symbols. Tokens need not be semantic units.
- A stochastic language or sequence model specifies a stochastic process whose state space is its vocabulary. An autoregressive model specifies its finite-sequence distributions through next-token conditionals.
- A decoder transformer maps a prefix to a categorical distribution by embedding tokens, applying causally masked attention and positionwise transformations, and using a softmax readout.
- Absolute position embeddings are added once at the input. RoPE and relative-position biases instead modify the attention calculation in every layer that uses them.
- The learned conditional distribution and the decoding algorithm are different objects. An argmax is optimal for a single zero–one-loss decision, but repeated tokenwise argmax does not solve a general sequence-level problem and is a poor default for open-ended generation.
- A full dense-attention pass on a length-t prefix has quadratic arithmetic cost. During sequential generation, a KV cache avoids recomputing old layer-position activations and changes total attention work from cubic to quadratic in the generated length. Old queries have no future use.
- Maximum context length is an input-capacity limit. Performance on some retrieval and reasoning tasks may deteriorate before the input reaches this limit. With bounded attention scores, a single softmax head cannot remain sharply concentrated as the context grows.

- The parameter vector θ contains the learned weights of the conditional model. It does not contain the current activations, KV cache, tokenizer, decoding rule, or harness. This lecture did not yet explain why learning produces useful values of θ.

1.12 Glossary

Harness; task state
The program surrounding the model, and the information that this program maintains between model calls.

Base alphabet; vocabulary; token
The symbols used to represent raw sequences; the finite collection of strings made available to the model; and one vocabulary item in a tokenized sequence.

Tokenizer; detokenizer
The map from a base-symbol string to vocabulary indices, and its inverse concatenation map.

Sequence orientation; matrix layout
Positions run left to right in the board diagrams. In the equations, positions are stacked as rows and the columns index coordinates such as vocabulary items or learned features.

Stochastic sequence model; autoregressive model
A probability law on token sequences, and its specification through conditional next-token distributions.

Embedding; logit; softmax readout
The vector assigned to a token, an unnormalized output score, and the map from the output scores to a categorical distribution.

Parameter; activation; layer-position activation; residual stream
A learned number shared across inputs; a computed value for the current input; and the vector h_i^ℓ carried at one sequence position between transformer layers.

Attention head; query; key; value; causal mask
One attention calculation, its three projected vectors, and the constraint that prevents a position from attending to future tokens.

Multi-head attention; residual connection; MLP; mixture of experts
Parallel attention heads; an identity path around a learned transformation; the dense positionwise sublayer; and its sparsely routed expert alternative.

Absolute position embedding; relative-position bias; RoPE
Three different mechanisms for making token order available to attention.

Decoding rule; sampling; greedy decoding
The procedure that turns a next-token distribution into an output token; random generation from that distribution; and tokenwise argmax.

Prefill; decode step; KV cache
The initial full-prompt forward pass; one incremental generation step; and the stored keys and values that prevent recomputation of old activations.

Maximum context length; attention entropy
The token-capacity ceiling for one model call, and a measure of how diffuse an attention distribution is.

A Supplement: how tokenizers are constructed

This appendix was not covered in the lecture. It records optional background, separate from the lecture narrative reconstructed above.

Two common corpus-based constructions illustrate the main ideas.

1. **Byte-pair encoding (BPE).** Start with base symbols, count adjacent token pairs in the training corpus, merge a most frequent pair into a new vocabulary item, and repeat until the desired vocabulary size is reached. The learned merge order determines the segmentation of new text [Sennrich et al., 2016].

2. **Unigram tokenization.** Start with a large collection of candidate substrings and probabilities $q(v)$. A segmentation is scored by

$$\prod_{t=1}^{T} q(v_{x_t}), \qquad \text{or equivalently} \qquad \sum_{t=1}^{T} \log q(v_{x_t}). \tag{1.39}$$

Fit the probabilities to the corpus, remove candidates whose deletion hurts the likelihood least, and repeat. At encoding time one may choose a highest-scoring segmentation or sample among segmentations [Kudo, 2018; Kudo and Richardson, 2018].

B Supplement: codebases for experiments

This appendix was not covered in the lecture. Each entry supports a different kind of experiment; students are not expected to install all of them.

NumPy RASP-L
A small reference implementation for writing and testing causal RASP-L sequence programs: https://github.com/apple/ml-np-rasp.

RASP and Tracr
The original RASP interpreter displays selectors and intermediate sequence operations. Tracr compiles a subset of RASP into concrete transformer weights: https://github.com/tech-srl/RASP and https://github.com/google-deepmind/tracr.

nanochat
A compact end-to-end codebase containing tokenization, pretraining, fine-tuning, evaluation, and inference. It includes small CPU and Apple-Silicon configurations, although training a capable language model still requires substantial computation: https://github.com/karpathy/nanochat.

Hugging Face Transformers
A broad model-definition and checkpoint ecosystem, useful for loading models and comparing tokenizers, architectures, and generation rules: https://github.com/huggingface/transformers.

TransformerLens
An inspection library for GPT-style models, useful for recording activations and attention patterns or intervening inside a forward pass: `https://github.com/TransformerLensOrg/TransformerLens`.

llama.cpp
A local inference implementation that makes tokenization, quantization, decoding, and key–value-cache behavior accessible to experiments: `https://github.com/ggml-org/llama.cpp`.

Bibliographic remarks

The compiler analogy is literal at the level of the interface: a lexer maps a character stream into tokens, although compiler tokens are specified by the programming language rather than induced from corpus frequencies; see Aho et al. [2006]. The supplementary appendix describes the BPE construction of Sennrich et al. [2016], the unigram formulation of Kudo [2018], and the SentencePiece system of Kudo and Richardson [2018].

The transformer architecture was introduced by Vaswani et al. [2017]. Their model combined encoder and decoder stacks; the equations in this lecture are a causal decoder specialization and use a pre-normalized block.

Three early pretraining recipes made the architectural alternatives concrete. The original GPT work paired a causal decoder language model with task-specific supervised fine-tuning [Radford et al., 2018]. BERT instead pretrained a bidirectional encoder with a masked-token objective and then fine-tuned its representations [Devlin et al., 2019]. T5 retained the encoder–decoder architecture and expressed many tasks through one text-to-text interface [Raffel et al., 2020]. GPT-3 later demonstrated that a sufficiently large causal language model could use task descriptions and examples supplied in its context, without a gradient update at test time [Brown et al., 2020]. These are historically important successful recipes; they do not establish that one of the three architectural forms uniformly dominates the others.

The original encoder–decoder split was designed for conditional sequence-to-sequence tasks such as translation. A fully visible encoder turns the complete source sequence into a memory; a causal decoder processes the target generated so far and uses a separate cross-attention sublayer to read that memory. In the causal decoder-only model described here, source or prompt and continuation occupy one sequence and pass through one shared stack. This is a natural fit when raw text has no privileged source–target split, and it gives pretraining, prompting, and continuation a homogeneous implementation. There is also a concrete attention-cost advantage when the encoder is bidirectional and the decoder-only model is causal. For source length S and target length T, compare one bidirectional encoder attention layer followed by one causal decoder attention layer and cross-attention, with one causal decoder-only attention layer on the concatenated source and target. The numbers of query–key scores are

$$N_{\text{enc-dec}} = S^2 + \frac{T(T+1)}{2} + ST, \tag{1.40}$$

$$N_{\text{dec-only}} = \frac{(S+T)(S+T+1)}{2} = \frac{S(S+1)}{2} + \frac{T(T+1)}{2} + ST. \tag{1.41}$$

The decoder-only layer saves $S(S-1)/2$ source–source scores: the upper triangle used by the bidirectional encoder is absent from causal attention. An implementation realizes this saving only when its causal-attention kernel avoids computing masked scores. This score count does not settle a whole-model comparison because depth, width, parameter count, projections, feed-forward

layers, and implementation also matter. Encoder–decoder models retain a strong inductive bias for translation and other fixed conditional tasks. In the controlled comparison by Raffel et al. [2020], the encoder–decoder model was strongest at roughly matched total computation; the paper also explains why parameter count and computation cannot both be matched by merely changing the number of layers.

Key–value caching also applies to encoder–decoder inference. The encoder output is computed once for a fixed source. Each decoder layer can cache the key and value projections of that source for cross-attention, while its causal self-attention cache grows with the generated target. At a decode step only the current queries are needed; old queries do not participate in future attention calculations.

Recent publicly released model families usually preserve the causal decoder-only interface while changing the surrounding recipe and internal details. Llama 3 is a dense example [Grattafiori et al., 2024], Mistral 7B uses grouped-query and sliding-window attention [Jiang et al., 2023], and DeepSeek-V3 uses a mixture-of-experts architecture [DeepSeek-AI, 2024]. The careful term is *open-weight*: releasing numerical weights does not by itself release the training data and code or grant every freedom conventionally associated with open-source software.

The original paper used additive sinusoidal position vectors. Rotary position embeddings are due to Su et al. [2021]; their position-dependent rotations of queries and keys expose relative displacement in the attention score. The bucketed learned relative-position bias in (1.12) follows Raffel et al. [2020]. Press et al. [2022] propose ALiBi, which instead adds a fixed head-dependent linear penalty based on relative distance.

The key–value cache is the standard implementation of incremental transformer decoding. Shazeer [2019] identified repeatedly loading cached keys and values as a memory-bandwidth bottleneck. In ordinary multi-head attention, each of H query heads has its own key and value head. Multi-query attention keeps the H query heads but shares one key head and one value head, reducing the per-layer cache from $2tHd_h$ to $2td_h$ scalars at prefix length t. This targets decoding, where the old keys and values must be read repeatedly; it does not give the analogous factor-H reduction in prefill arithmetic. Shazeer's experiments found a large decoding-speed improvement with a small quality loss. Grouped-query attention interpolates between the two designs by using G shared key–value groups, giving a cache of $2tGd_h$ scalars [Ainslie et al., 2023]; its use in Mistral 7B and Llama 3 makes the idea part of modern practice. Thus the Shazeer reference matters both historically and as the origin of a current KV-cache tradeoff.

Dao et al. [2022] introduced FlashAttention, which computes exact dense attention by tiling and an online softmax. This reduces memory traffic and avoids materializing the full attention matrix, but does not remove the quadratic arithmetic of exact dense attention. The course returns to this distinction in Lecture 27.

The bounded-score calculation left for homework is elementary. Chiang and Cholak [2022] study a related length-dependent loss of confidence and show that the obstruction depends on architectural details; one of their remedies scales attention logits with $\log T$. This is useful context for the homework result, which is not a general explanation of long-context failures.

The experiments of Liu et al. [2024] vary the position of relevant information in multi-document question answering and key–value retrieval, demonstrating the distinction between accepting a long context and using it reliably. Hong et al. [2025] use the term "context rot" for performance degradation as input length grows across a broader collection of tasks and models. Both works study complete trained systems. The bounded-score calculation supplies one possible mechanism, but does not establish that it causes the observed degradation; the extent of any connection is unclear.

RASP is a small language for writing sequence computations using operations chosen to resemble transformer primitives. Weiss et al. [2021] use it to make attention-based constructions explicit.

Zhou et al. [2024] introduce RASP-L for causal decoder-only transformers and propose that short RASP-L programs help predict which algorithmic tasks a trained transformer will learn in a length-generalizing way. This is a conjectural empirical organizing principle, not a theorem that gradient training recovers every short program. The papers are particularly useful because they force a proposed computation to be written as specific comparisons, causal selections, aggregations, and position operations. The NumPy implementation and Tracr compiler are listed in Appendix B.

References

A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison–Wesley, second edition, 2006.

R. Sennrich, B. Haddow, and A. Birch. Neural Machine Translation of Rare Words with Subword Units. *ACL*, 2016. https://arxiv.org/abs/1508.07909.

T. Kudo. Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates. *ACL*, 2018. https://arxiv.org/abs/1804.10959.

T. Kudo and J. Richardson. SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing. *EMNLP System Demonstrations*, 2018. https://aclanthology.org/D18-2012/.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, L. Kaiser, and I. Polosukhin. Attention Is All You Need. *NeurIPS*, 2017. https://arxiv.org/abs/1706.03762.

A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever. Improving Language Understanding by Generative Pre-Training. OpenAI technical report, 2018. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.

J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL*, 2019. https://arxiv.org/abs/1810.04805.

T. B. Brown et al. Language Models are Few-Shot Learners. *NeurIPS*, 2020. https://arxiv.org/abs/2005.14165.

A. Grattafiori et al. The Llama 3 Herd of Models. 2024. https://arxiv.org/abs/2407.21783.

A. Q. Jiang et al. Mistral 7B. 2023. https://arxiv.org/abs/2310.06825.

DeepSeek-AI. DeepSeek-V3 Technical Report. 2024. https://arxiv.org/abs/2412.19437.

J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. 2021. https://arxiv.org/abs/2104.09864.

C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. https://www.jmlr.org/papers/v21/20-074.html.

O. Press, N. A. Smith, and M. Lewis. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. *ICLR*, 2022. https://arxiv.org/abs/2108.12409.

N. Shazeer. Fast Transformer Decoding: One Write-Head is All You Need. 2019. https://arxiv.org/abs/1911.02150.

J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *EMNLP*, 2023. https://arxiv.org/abs/2305.13245.

T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS*, 2022. https://arxiv.org/abs/2205.14135.

D. Chiang and P. Cholak. Overcoming a Theoretical Limitation of Self-Attention. *ACL*, 2022. https://aclanthology.org/2022.acl-long.527/.

N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 2024. https://arxiv.org/abs/2307.03172.

K. Hong, A. Troynikov, and J. Huber. Context Rot: How Increasing Input Tokens Impacts LLM Performance. Chroma technical report, 2025. https://www.trychroma.com/research/context-rot.

G. Weiss, Y. Goldberg, and E. Yahav. Thinking Like Transformers. *ICML*, 2021. https://proceedings.mlr.press/v139/weiss21a.html.

H. Zhou, A. Bradley, E. Littwin, N. Razin, O. Saremi, J. Susskind, S. Bengio, and P. Nakkiran. What Algorithms Can Transformers Learn? A Study in Length Generalization. *ICLR*, 2024. https://arxiv.org/abs/2310.16028.